Звіт до лабораторної роботи №2
Дослідницька частина
Python

1) Після копіювання репозиторію я спочатку зробив pip freeze і отримав файл backend.txt з фіксованими залежностями, який і буду використовувати далі. Після цього я написав перший Dockerfile (лінк). Він не оптимізований, бо одразу копіює всі файли, а потім встановлює залежності, тобто не використовує функціонал кешування, що можна побачити при спробі змінити код:

mywebapp:v1
build time: 53s
size: 406MB
rebuild time: 16s

2) Щоб мати frame of reference, я написав другий Dockerfile (лінк). Він вже повністю використовує функціонал кешування: спочатку копіює та встановлює залежності, потім копіює код. Тобто ми не перевстановлюємо залежності при кожній зміні коду, що відображається на часі перезбірки:

mywebapp:v2
build time: 46s
size: 406MB
rebuild time: 6s

3) Наступний Dockerfile (лінк) замість debian-based python використаємо alpine-based. Він менший за розміром, що і можна побачити:

mywebapp:v3
build time: 44s
size: 142MB
rebuild time: 6s

4) Далі я додав бібліотеку numpy у вигляді ендпойнту (лінк). Спочатку створимо Dockerfile (лінк) з debian python:

mywebapp:v4
build time: 4m35s
size: 439MB
rebuild time: 6s

Як бачимо, час збірки збільшився на +-4 хвилини, розмір збільшився на 30МБ.

5) Далі те саме, але з alpine (лінк):

mywebapp:v5
build time: 5m43s
size: 243MB
rebuild time: 9s

Як бачимо, час збірки збільшився на +-5 хвилини, що помітно більше ніж debian, і розмір збільшився на 100МБ, що набагато більший приріст у відсотковому відношенні.

Висновки:
debian:
- більший розмір
- більш ефективний з математичними бібліотеками (завдяки наявності готових wheels)

alpine:
- менший розмір
- неефективний з математичними бібліотеками (компілює numpy самостійно)

Час перезбірки залежить не від розміру зображення, а від якості Dockerfile.

DNS

Спочатку я створив окрему docker-мережу для ізольованого тестування:

```
docker network create dns-lab
```

Далі я запустив простий DNS-сервер на базі dnsmasq у контейнері Alpine. У конфігурації я явно задав правило резолвінгу для домену myservice.internal.corp, який має повертати IP-адресу 10.0.0.50:

```
docker run --rm -it --name dns-server --network dns-lab \
alpine sh -c "apk add dnsmasq && \
echo 'address=/myservice.internal.corp/10.0.0.50' > /etc/dnsmasq.conf && \
dnsmasq -k --log-queries --log-facility=-"
```

Цей сервер також виводить усі DNS-запити в лог, що дозволяє детально проаналізувати поведінку клієнтів.

Після цього я протестував резолвінг у контейнері на базі Ubuntu (glibc), використовуючи getent hosts та передаючи параметр --dns-search="corp":

```
docker run --rm --network dns-lab \
--dns=$(docker inspect -f '{{range
.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
--dns-search="corp" \
ubuntu:latest getent hosts myservice.internal
```

У логах DNS-сервера я побачив наступну поведінку:

```
query[AAAA] myservice.internal → NXDOMAIN
query[AAAA] myservice.internal.corp → NODATA-IPv6
query[A] myservice.internal → NXDOMAIN
query[A] myservice.internal.corp → 10.0.0.50
```

Це означає, що система спочатку намагалася знайти запис для myservice.internal, але після невдачі автоматично застосувала search-домен corp і виконала запит до myservice.internal.corp, який вже успішно зарезолвився.

Далі я виконав аналогічний тест у контейнері Alpine (musl):

```
docker run --rm --network dns-lab \
--dns=$(docker inspect -f '{{range
.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
--dns-search="corp" \
alpine:latest getent hosts myservice.internal
```

У логах DNS-сервера було видно:

```
query[AAAA] myservice.internal → NXDOMAIN
query[A] myservice.internal → NXDOMAIN
```

У цьому випадку контейнер навіть не намагався звернутися до myservice.internal.corp. Тобто search-домен фактично не був застосований.

Отримані результати показують принципову різницю між glibc та musl:

glibc (Ubuntu):
- коректно обробляє параметр search
- після невдалого запиту автоматично доповнює домен
- виконує повторні запити з урахуванням search-домену

musl (Alpine):
- не застосовує search-домен у цьому сценарії
- виконує лише прямий запит до переданого імені
- не робить повторних спроб із розширеним доменом

Причина такої поведінки полягає в реалізації DNS-резолвера:

- glibc має повноцінну реалізацію логіки /etc/resolv.conf, включаючи search та ndots
- musl використовує спрощений резолвер, який не повністю підтримує ці механізми

Можу зробити припущення, що така поведінка може призводити до серйозних проблем у реальних системах:

- сервіси можуть не знаходити один одного за короткими іменами
- можуть виникати складні для діагностики помилки в Docker Compose або Kubernetes
- одна і та ж система може працювати в Debian-контейнері, але не працювати в Alpine

Golang

1) Після клонування репозиторію я спочатку створив базовий Dockerfile (single-stage) (лінк), який виконує збірку застосунку всередині контейнера та одразу запускає його. У цьому варіанті в контейнер копіюється весь код, залежності, а також залишається весь toolchain Go.
Результат:

v1:
time: 2m11s
size: 520MB

Аналіз через dive (я прибрав пусті директорії) :

```
9.2 MB  ├── app
60 kB   │   ├─⊕ .git
2.4 kB  │   ├── .gitignore
119 B   │   ├── Dockerfile
1.1 kB  │   ├── README.rst
9.0 MB  │   ├── app
2.5 kB  │   ├── cmd
664 B   │   │   ├── query.go
493 B   │   │   ├── root.go
1.3 kB  │   │   └── serve.go
181 B   │   ├── go.mod
76 kB   │   ├── go.sum
214 B   │   ├── lib
214 B   │   │   └── fizzbuzz.go
119 B   │   ├── main.go
447 B   │   └── templates
447 B   │       └── index.html
5.5 MB  ├─⊕ bin
499 kB  ├─⊕ etc
610 MB  ├─⊕ go
9.5 MB  ├─⊕ lib
3.7 MB  ├─⊕ root
4.0 MB  ├─⊕ sbin
909 MB  ├─⊕ usr
7.2 MB  └─⊕ var
```

Присутній весь Go toolchain (/go, /usr, /bin), вихідний код проєкту (.git, .go файли), кеші, системні бібліотеки та інше. Більшість файлів не потрібні для запуску застосунку, оскільки для runtime достатньо лише скомпільованого бінарного файлу та статичних ресурсів.

2) Далі я реалізував multi-stage build із використанням scratch (лінк). На першому етапі відбувається збірка бінарного файлу, а на другому копіюється лише результат збірки. Спочатку виникла помилка “no such file or directory”, бо бінарник був динамічно зібраний і очікував наявність системних бібліотек. Я додав цей параметр збірки:

```
CGO_ENABLED=0 go build -o app
```

Це змусило Go створити статично скомпільований бінарник, який не залежить від libc. Потім я знову отримав цю помилку, але вже з іншої причини: я копіював лише бінарник, і він намагався імпортувати index.html, який я не додав. Після додавання все запрацювало.
Результат:

v2
time: 1m26s
size: 4.76MB

Аналіз через dive:

```
8.9 MB  └── app
8.9 MB      ├── app
447 B       └── templates
447 B           └── index.html
```

В образі є лише скомпільований бінарник та папка templates з index.html. Це означає, що образ містить тільки необхідні файли для запуску.

Особливості scratch-образу:

- мінімальний розмір
- відсутність shell
- неможливо виконати docker exec
- складно дебажити

3) Після цього я реалізував multi-stage build Dockerfile (лінк) із використанням distroless образу (static-debian13).

Результати:

v3
time: 1m29s
size: 5.58MB

Аналіз через dive (знову прибрав пусті директорії):

```
8.9 MB  ├── app
8.9 MB  │   ├── app
447 B   │   └── templates
447 B   │       └── index.html
323 kB  ├─⊕ etc
1.8 MB  ├─⊕ usr
71 kB   └─⊕ var
```

В образі є лише бінарник застосунку, index.html та мінімальний набір системних файлів (/etc, /usr). У порівнянні зі scratch він має трохи більший розмір, але присутнє базове середовище виконання.

Висновки:
naive (single-stage):

- дуже великий розмір
- містить багато зайвих файлів (toolchain, кеші, вихідний код)
- простий у використанні

scratch:

- мінімальний розмір
- містить лише необхідні файли
- потребує статичної компіляції
- незручний для дебагу та роботи

distroless:

- майже такий самий малий розмір, як scratch
- містить мінімальне runtime-середовище
- більш практичний у використанні
- оптимальний баланс між розміром і зручністю

Multi-stage збірка значно зменшує розмір образу та прибирає зайві залежності, а використання distroless є найкращим компромісом між мінімалізмом і зручністю експлуатації.

Практична частина

Після виконання дослідницької частини я перейшов до контейнеризації застосунку з попередньої лабораторної роботи, який складався з трьох сервісів: web application (Java), database (MariaDB) та nginx як reverse proxy.

Початковий підхід

Спочатку я орієнтувався на bash-скрипт із попереднього завдання, де:

- база даних встановлювалась локально
- застосунок запускався як systemd-сервіс
- nginx працював на тій самій машині

Очевидно, що такий підхід не переноситься напряму в Docker, тому я розбив систему на окремі сервіси:

- db - MariaDB
- app - Java застосунок
- nginx - reverse proxy

Побудова docker-compose

Я створив файл docker-compose.yml (лінк), у якому:

- кожен сервіс запускається в окремому контейнері
- використовується окрема мережа:

```
networks:
  mynetwork:
    driver: bridge
```

Це дозволяє контейнерам взаємодіяти через DNS-імена (db, app), а не через localhost.

Контейнеризація бази даних

Для бази даних я використав готовий образ mariadb:12.

Спочатку я намагався виконувати SQL-міграції з контейнера застосунку через mysql клієнт, але це призвело до проблем:

- в контейнері не було mysql-клієнта
- з'являлась залежність від shell

У підсумку я перейшов до правильнішого підходу - використання механізму ініціалізації MariaDB:

```
volumes:
  - ./migrate.sql:/docker-entrypoint-initdb.d/init.sql
```

Це дозволяє:

- автоматично виконати SQL при першому старті БД
- повністю прибрати міграції з контейнера застосунку

Також я додав healthcheck:

```
healthcheck:
  test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
```

Це дозволяє іншим сервісам чекати готовності БД.

Контейнеризація застосунку
Спочатку я використовував простий Dockerfile, який:

1. копіював готовий .jar
2. запускав його через java -jar

Але такий підхід:

- не є відтворюваним
- залежить від локального білду

Тому я перейшов до multi-stage build (Dockerfile.app):

```
FROM maven:3.9-eclipse-temurin-17 AS builder
…
RUN mvn package -DskipTests
```

Це дозволяє:

- збирати застосунок прямо в Docker
- отримати однаковий результат на будь-якій машині

Далі я використав distroless образ:

```
FROM gcr.io/distroless/java17-debian13
```

У фінальному образі:

- немає shell
- немає package manager
- є тільки runtime для запуску Java

Оптимізація збірки
Для пришвидшення збірки я використав кешування залежностей:

```
COPY pom.xml .
RUN mvn dependency:go-offline
```

Це означає:

- залежності завантажуються один раз
- при зміні коду повторно не скачуються

Також я додав .dockerignore:

```
*
!pom.xml
!src
!migrate.sql
!nginx.conf
```

Це дозволило:

- мінімізувати build context
- пришвидшити збірку
- уникнути передачі зайвих файлів

Контейнеризація nginx
Для nginx я використав офіційний образ та конфіг (лінк):
proxy_pass http://app:3000;

Важливий момент:
- замість 127.0.0.1 використовується ім'я сервісу app
- працює через внутрішній DNS Docker

Основні труднощі
1. *Використання localhost*
Спочатку Java застосунок використовував:
ipAddress("127.0.0.1");
String url = "jdbc:mariadb://localhost:3306/mywebapp";

У Docker це не працює, оскільки:
- localhost означає поточний контейнер
- сервіси знаходяться в різних контейнерах

Рішення:
ipAddress("0.0.0.0");
String url = "jdbc:mariadb://db:3306/mywebapp";

2. *Запуск міграцій*
Початковий підхід:
запуск міграцій з app контейнера через mysql

Проблеми:
- потрібен mysql client
- складний startup
- залежність від shell

Фінальне рішення:
- ./migrate.sql:/docker-entrypoint-initdb.d/init.sql
- повна ізоляція логіки

3. *Проблеми з портами*
nginx не міг підключитись до app через:
connection refused

Причина:
застосунок слухав тільки 127.0.0.1

Рішення:
заміна на 0.0.0.0

Отримані результати

app	61.7MB
db	111MB
nginx	65.8MB

Основне зменшення розміру було досягнуто за рахунок:

- переходу з JDK на JRE
- використання distroless
- multi-stage build

Час збірки

- перша збірка - довша (завантаження залежностей Maven)
- повторна - значно швидша завдяки кешу

Висновки та рекомендації

- Контейнери повинні бути ізольованими
    a. не використовувати localhost
    b. використовувати DNS-імена сервісів
- Multi-stage build - обов'язковий
    a. дозволяє відокремити build і runtime
    b. значно зменшує розмір образу
- Distroless - хороший вибір для production
    a. мінімальний розмір
    b. підвищена безпека
- Міграції не повинні виконуватись в app контейнері
    a. краще використовувати init-скрипти БД або окремі job-и
- Healthchecks важливі
    a. дозволяють коректно керувати залежностями між сервісами
- .dockerignore критично важливий
    a. зменшує build context
    b. пришвидшує збірку

У результаті було отримано повністю контейнеризовану систему, яка:

- запускається однією командою
- ізольована
- відтворювана
- оптимізована за розміром і структурою